

1 Introdução e Contextualização do Sistema

O mercado de apostas esportivas (*iGaming* e *Sportsbooks*) demanda infraestruturas de altíssimo desempenho e baixíssima latência. Durante partidas ao vivo, ocorrências como gols, cartões vermelhos ou alterações súbitas na probabilidade de vitória exigem reavaliações instantâneas das cotações (*Odds*). Modelos tradicionais baseados em requisições HTTP REST convencionais falham nesse cenário devido ao custo de abertura de conexões TCP e overhead de cabeçalhos HTTP para cada atualização.

A adoção do protocolo **WebSocket** provê um canal Full-Duplex persistente sobre uma única conexão TCP. Isso permite que o servidor envie atualizações contínuas de cotações em tempo real para milhares de clientes simultâneos sem necessidade de *polling*, além de viabilizar a recepção imediata dos cupons de aposta submetidos pelos usuários e a recalculação em tempo real do valor de encerramento antecipado (**Cash Out**).

Esta atividade prática orienta a construção de um ecossistema completo de apostas em tempo real, integrando painéis dinâmicos de cotações, transmissão de estatísticas de jogo e mecanismo de apostas bidirecional.

Plataforma de Bets em Tempo Real com WebSocket

Visão Geral do Projeto

As plataformas modernas de apostas esportivas ao vivo baseiam-se em fluxos bidirecionais contínuos de altíssima velocidade para atualizar cotações (*Odds*) à medida que o jogo se desenrola. O objetivo desta atividade é implementar uma aplicação Web simulação de apostas ao vivo utilizando o protocolo **WebSocket**. Cada participante receberá um saldo fictício inicial de R\$ 100,00, poderá submeter apostas em mercados dinâmicos, acompanhar flutuações de cotações, receber notificações de gols e utilizar o recurso avançado de **Cash Out** pré-jogo/em-jogo.

2 Especificação Completa de Requisitos

2.1 Requisitos Funcionais

[RF-01] Atribuição de Saldo Fictício Inicial: Ao estabelecer com sucesso a conexão bidirecional via WebSocket, o servidor deve vincular à sessão do apostador um saldo fictício inicial no valor exato de R\$ 100,00, transmitindo a confirmação da carteira no evento inicial de handshake.

Prioridade: Crítica

[RF-02] Transmissão Dinâmica de Odds (ODDS_UPDATE): O servidor backend deve simular o andamento da partida ao vivo e atualizar periodicamente (a cada 2 a 3 segundos) as cotações dos mercados de aposta (*1X2*, *Próximo Gol*, *Total de Gols*), transmitindo o broadcast do evento ODDS_UPDATE em JSON para todos os clientes conectados.

Prioridade: Crítica

[RF-03] Submissão e Validação de Apostas (PLACE_BET): O cliente deve permitir a montagem e envio do bilhete de aposta contendo o ID da partida, mercado escolhido, opção selecionada, odd aceita e valor em Reais. O servidor deve verificar a disponibilidade de saldo em memória, debitar o valor da aposta atômica e emitir o comprovante `BET_CONFIRMED`.

Prioridade: Crítica

[RF-04] Cálculo e Atualização de Cash Out em Tempo Real: Durante a realização da partida ao vivo, o servidor deve recalcular continuamente o valor de resgate antecipado (*Cash Out*) para cada aposta ativa com base na probabilidade instantânea do evento, transmitindo os novos valores de encerramento para o painel do usuário.

Prioridade: Alta

[RF-05] Processamento e Encerramento Antecipado (CASH_OUT): Ao acionar o botão de Cash Out no painel, o navegador deve enviar o evento `CASH_OUT` com o ID do bilhete. O servidor deve encerrar imediatamente a aposta ativa, creditar o valor acordado no saldo do apostador e responder com a mensagem `CASH_OUT_CONFIRMED`.

Prioridade: Alta

[RF-06] Notificação Instantânea de Eventos do Jogo (GOAL): Quando ocorrer um evento decisivo na simulação da partida (ex: Gol ou Cartão Vermelho), o servidor deve emitir um broadcast prioritário congelando temporariamente as apostas nos mercados afetados e exibindo um banner pop-up com sinal sonoro na tela dos clientes.

Prioridade: Alta

[RF-07] Liquidação Automática de Apostas Vencedoras: Ao término da partida ou encerramento do mercado, o servidor deve avaliar todas as apostas abertas, calcular o retorno potencial ($\text{Valor Apostado} \times \text{Odd}$) para as seleções vitoriosas, creditar os ganhos na carteira e emitir a atualização de saldo (`BALANCE_UPDATE`).

Prioridade: Crítica

[RF-08] Painel do Apostador e Ranking da Sessão: A interface front-end deve apresentar um painel consolidado exibindo o saldo atualizado, bilhetes de aposta ativos com botão de Cash Out dinâmico e uma tabela de ranking em tempo real destacando os apostadores com maiores saldos da sessão.

Prioridade: Média

2.2 Requisitos Não-Funcionais

[RNF-01] Prevenção de Race Condition no Cash Out: O servidor backend deve implementar mecanismos de concorrência e trava atômica. Se um evento de gol for registrado no exato milissegundo em que uma requisição de Cash Out for recebida, a solicitação de resgate deve ser rejeitada imediatamente para evitar liquidações inconsistentes.

Prioridade: Crítica

[RNF-02] Garantia de Integridade e Saldo Não Negativo: O sistema deve validar estritamente todas as operações de débito, rejeitando qualquer tentativa de aposta ou transação

cujo valor exceda o saldo corrente do apostador, impedindo saldos negativos.

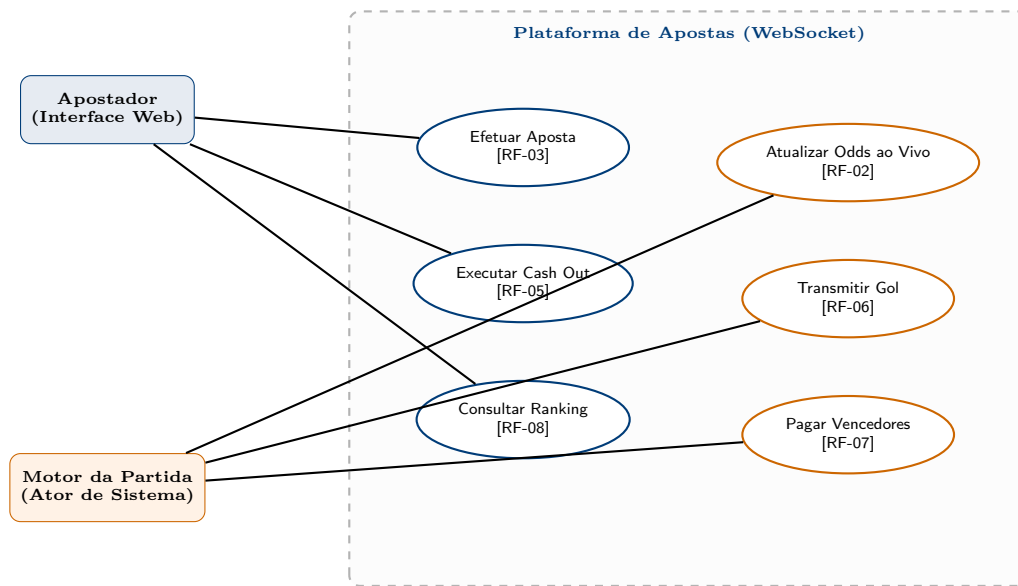
Prioridade: Crítica

[RNF-03] Altíssima Eficiência e Baixa Latência (< 100 ms): A entrega de cotações atualizadas e o processamento dos bilhetes de aposta devem ser trafegados em estruturas JSON compactas via WebSocket, com tempo de resposta inferior a 100 ms.

Prioridade: Alta

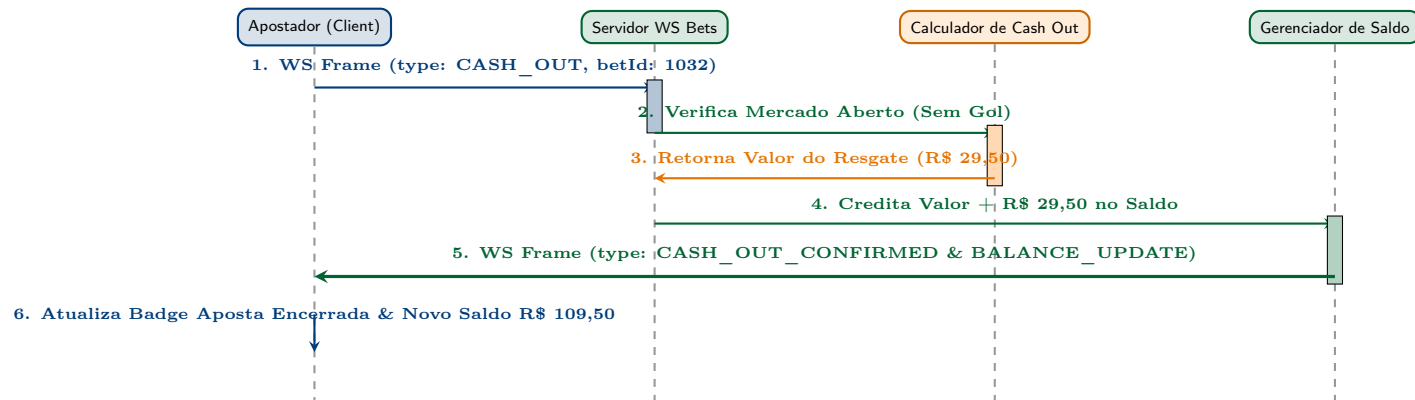
3 Diagrama de Casos de Uso (TikZ UML)

Modelagem UML dos casos de uso da plataforma de apostas esportivas.



4 Diagrama de Sequência de Cash Out (TikZ UML)

Sequência detalhando o fluxo atômico de solitação de Cash Out, validação no servidor e crédito no saldo.



As comunicações em tempo real da plataforma de apostas ocorrem via conexão WebSocket bidirecional (`ws://localhost:3000`). Os dados trafegam no formato **JSON**.

4.1 Eventos Enviados pelo Cliente (Cliente → Servidor)

4.1.1 1. Submissão de Aposta (PLACE_BET)

Enviado quando o apostador confirma um bilhete na interface de apostas.

type (*string*): Comando da mensagem ("PLACE_BET").

matchId (*integer*): Identificador único da partida esportiva.

mercadoId (*string*): Código do mercado selecionado (ex: "VENCEDOR_1X2").

selecao (*string*): Opção escolhida pelo usuário ("CASA", "EMPATE" ou "FORA").

odd (*float*): Cotação aceita pelo usuário no momento da aposta.

valor (*float*): Valor em Reais (R\$) a ser debitado do saldo fictício.

```

1 {
2   "type": "PLACE_BET",
3   "matchId": 15,
4   "mercadoId": "VENCEDOR_1X2",
5   "selecao": "CASA",
6   "odd": 1.90,
7   "valor": 20.00
8 }
  
```

Listing 1: Payload JSON para submissão de aposta (PLACE_BET)

4.1.2 2. Solicitação de Encerramento Antecipado (CASH_OUT)

Descrição do Evento enviado pelo Cliente: Enviado quando o usuário clica no botão "Cash Out" para antecipar o valor de resgate oferecido pelo algoritmo da plataforma.

type (*string*): Comando da mensagem ("CASH_OUT").

betId (*integer*): Identificador único do bilhete de aposta ativo.

matchId (*integer*): ID da partida correspondente.

requestedValue (*float*): Valor de encerramento exibido em tela e aceito pelo apostador.

```
1 {  
2   "type": "CASH_OUT",  
3   "betId": 1032,  
4   "matchId": 15,  
5   "requestedValue": 29.50  
6 }
```

Listing 2: Payload JSON para solicitacao de Cash Out (CASH_OUT)

4.2 Eventos de Broadcast do Servidor (Servidor → Clientes)

4.2.1 1. Atualização de Cotações em Tempo Real (ODDS_UPDATE)

Descrição do Broadcast transmitido pelo Servidor: Disparado periodicamente para todos os apostadores conectados quando os eventos de campo (faltas, posse de bola, ataques) alteram a probabilidade dos mercados.

event (*string*): Nome do evento em broadcast ("ODDS_UPDATE").

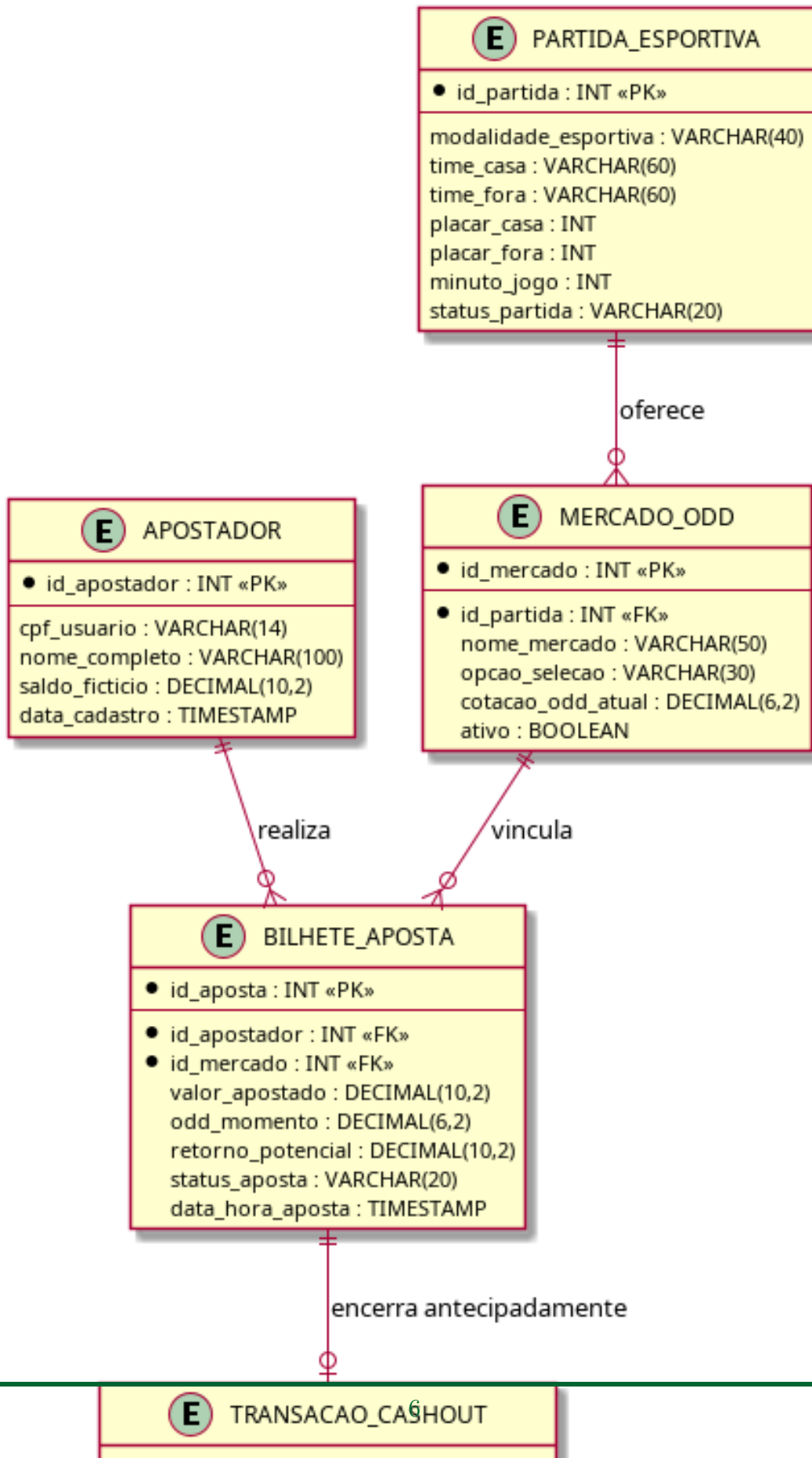
matchId (*integer*): ID da partida com cotações alteradas.

odds (*object*): Mapa contendo os novos multiplicadores para cada mercado.

timestamp (*string*): Data/hora ISO-8601 da atualização das cotações.

5 Diagrama Entidade-Relacionamento do Banco de Dados (PlantUML ERD)

O funcionamento de uma plataforma de apostas esportivas em tempo real baseia-se em um modelo relacional para gerenciar apostadores, partidas ao vivo, mercados de cotações (*Odds*), bilhetes de aposta e transações de *Cash Out*. O diagrama a seguir, desenvolvido em **PlantUML**, descreve a estrutura do banco de dados.



5.1 Detalhamento das Entidades e Atributos

APOSTADOR: Cadastra os usuários e gerencia o saldo financeiro fictício.

- `id_apostador` (*INT*, Chave Primária): Identificador único do apostador.
- `cpf_usuario` (*VARCHAR(14)*): Documento do cliente.
- `nome_completo` (*VARCHAR(100)*): Nome completo do usuário.
- `saldo_ficticio` (*DECIMAL(10,2)*): Carteira de saldo em Reais (ex: R\$ 100,00).
- `data_cadastro` (*TIMESTAMP*): Instante de registro na plataforma.

PARTIDA_ESPORTIVA: Armazena os eventos esportivos transmitidos em tempo real.

- `id_partida` (*INT*, Chave Primária): Identificador da partida.
- `modalidade_esportiva` (*VARCHAR(40)*): Esporte ("FUTEBOL", "BASQUETE").
- `time_casa` (*VARCHAR(60)*): Nome do clube mandante.
- `time_fora` (*VARCHAR(60)*): Nome do clube visitante.
- `placar_casa` (*INT*): Gols/Pontos do time mandante.
- `placar_fora` (*INT*): Gols/Pontos do time visitante.
- `minuto_jogo` (*INT*): Tempo decorrido da partida em minutos.
- `status_partida` (*VARCHAR(20)*): Condição ("AO_VIVO", "ENCERRADA", "SUSPENSA").

MERCADO_ODD: Tabela contendo as cotações flutuantes dos mercados de aposta.

- `id_mercado` (*INT*, Chave Primária): Identificador do mercado/odd.
- `id_partida` (*INT*, Chave Estrangeira): Partida vinculada.
- `nome_mercado` (*VARCHAR(50)*): Tipo ("1X2", "PROXIMO_GOL").
- `opcao_selecao` (*VARCHAR(30)*): Palpite ("CASA", "EMPATE", "FORA").
- `cotacao_odd_atual` (*DECIMAL(6,2)*): Multiplicador atualizado via WebSocket (ex: 1.80).
- `ativo` (*BOOLEAN*): Flag de mercado aberto/suspenso.

BILHETE_APOSTA: Registra cada cupom de aposta submetido pelos apostadores.

- `id_aposta` (*INT*, Chave Primária): Identificador único do bilhete.
- `id_apostador` (*INT*, Chave Estrangeira): Apostador titular do bilhete.
- `id_mercado` (*INT*, Chave Estrangeira): Mercado e seleção escolhida.
- `valor_apostado` (*DECIMAL(10,2)*): Valor em Reais investido na aposta.
- `odd_momento` (*DECIMAL(6,2)*): Odd travada no momento da confirmação.
- `retorno_potencial` (*DECIMAL(10,2)*): Valor do prêmio máximo (Valor × Odd).
- `status_aposta` (*VARCHAR(20)*): Situação ("ATIVA", "GREEN", "RED", "CASH_OUT").
- `data_hora_aposta` (*TIMESTAMP*): Carimbo de data/hora do registro.

TRANSACAO_CASHOUT: Registro de auditoria do encerramento antecipado de apostas.

- **id_cashout** (*INT*, Chave Primária): Identificador da transação.
- **id_aposta** (*INT*, Chave Estrangeira): Bilhete de aposta encerrado.
- **valor_resgate_oferecido** (*DECIMAL(10,2)*): Valor de cotação de resgate.
- **valor_resgatado** (*DECIMAL(10,2)*): Valor efetivamente creditado no saldo.
- **timestamp_efetivacao** (*TIMESTAMP*): Instante da conclusão da operação.

5.2 Relacionamentos e Regras de Integridade

- **PARTIDA_ESPORTIVA → MERCADO_ODD (1:N)**: Uma partida esportiva possui dezenas de opções e mercados de aposta.
- **APOSTADOR → BILHETE_APOSTA (1:N)**: Um apostador pode criar múltiplos bilhetes de aposta em partidas distintas.
- **MERCADO_ODD → BILHETE_APOSTA (1:N)**: Uma seleção de odd atrai bilhetes de apostadores diversos.
- **BILHETE_APOSTA → TRANSACAO_CASHOUT (1:1)**: Um bilhete de aposta pode ser encerrado antecipadamente no máximo uma vez.

6 Protótipos de Interface, Histórias de Usuário e Fluxos de Navegação

A seguir são apresentados os protótipos de interface responsivos desenvolvidos para a Plataforma de Bets Esportivas Ao Vivo em Tempo Real baseada em WebSockets (Ws2). Cada tela é acompanhada de sua especificação formal em formato de História de Usuário (Scrum), critérios de aceite, fluxo feliz, fluxos alternativos/exceções, detalhamento de botões e o mapeamento completo de navegação entre as telas.

6.1 Tela 1: Lobby de Apostas Ao Vivo (tela1-odds.html)

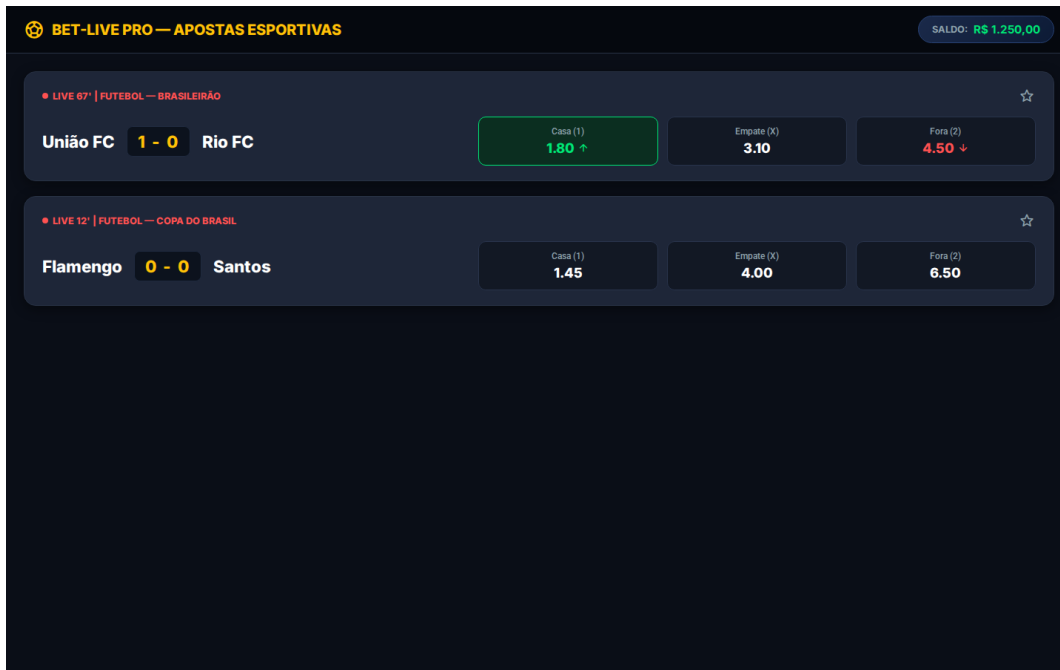


Figura 2: Protótipo do Lobby de Apostas — Visão Desktop (Cotações Flutuantes Ao Vivo).



Figura 3: Protótipo do Lobby de Apostas — Visão Mobile Responsiva.

6.1.1 História de Usuário (Scrum)

US-01: Acompanhamento de Cotações (Odds) e Seleção de Palpites

Como Apostador Esportivo,

Eu quero visualizar a lista de partidas ao vivo com cotações (*Odds*) atualizadas instantaneamente via WebSocket sem recarga de página,

Para que eu possa identificar oportunidades vantajosas de aposta de acordo com os acontecimentos do jogo.

6.1.2 Critérios de Aceite

- Exibir lista de partidas divididas por modalidades esportivas (Futebol, Basquete, E-Sports).
- Atualizar visualmente as cotações com animações em verde (subida de odd) e vermelho (queda de odd) via evento `ODDS_UPDATE`.

- Permitir o bloqueio instantâneo do mercado (*Mercado Suspenso*) quando lances decisivos ocorrerem no jogo.
- Adicionar a seleção ao cupom de apostas ao clicar em qualquer botão de odd.

6.1.3 Fluxo Feliz (Happy Path)

1. O apostador acessa a página principal `tela1-odds.html`.
2. O navegador conecta-se ao servidor de apostas no endpoint WebSocket `ws://localhost:8080/bets`.
3. O servidor transmite o broadcast `ODDS_UPDATE` alterando a cotação do Flamengo de 2.10 para 1.95.
4. O botão da odd pisca na cor verde e exibe o novo multiplicador 1.95.
5. O apostador clica na odd 1.95. A seleção é destacada e enviada para a caderneta de apostas (Bet Slip).

6.1.4 Fluxos Alternativos e Exceções

- **Fluxo Alternativo (Mercado Suspenso):** Ao ser marcado um pênalti na partida, o servidor envia o status de suspensão. Todos os botões de odds daquele jogo exibem o ícone de um cadeado e o aviso *"[Mercado Temporariamente Suspenso]"*.
- **Fluxo de Exceção (Queda de Conexão):** Caso a internet do apostador oscile, a barra de status exibe o aviso *"[Conexão WS Interrompida — Cotações Pausadas]"*.

6.1.5 Detalhamento de Botões e Componentes Interativos

Botões de Odds (Cotações): Exibe os multiplicadores de vitória (1X2). Clique insere o palpite no cupom.

Filtros por Esporte (Chips): Alterna a visualização entre Futebol, Basquete e Tênis.

Badge "Ao Vivo"(Pulsante): Indica partidas em andamento com tempo decorrido do jogo.

6.1.6 Navegação entre Protótipos

A seleção de uma odd nesta tela foca e expande a **Tela 2 (Bet Slip)** para preenchimento e confirmação do valor da aposta.

6.2 Tela 2: Detalhes da Partida e Bet Slip (tela2-betslip.html)

DETALHES DA PARTIDA & BILHETE DE APOSTA (BET SLIP)

União FC 1 × 0 Rio FC (67 min)

Posse de Bola

União 58% vs 42% Rio FC

Chutes a Gol

8 vs 2

Escanteios

5 vs 1

LIVE TICKER (WS FEED)

67' Gol do União FC! (Carlos Silva)

62' Cartão Amarelo para camisa 5 do Rio FC

BILHETE DE APOSTA (BET SLIP)

União FC (Vencedor)

Odd Seleccionada: 1.80

Valor da Aposta (R\$):

R\$ 50,00

Retorno Potencial:

R\$ 90,00

CONFIRMAR APOSTA [WS]

Figura 4: Protótipo de Bet Slip e Detalhes — Visão Desktop (Caderneta de Apostas).

12

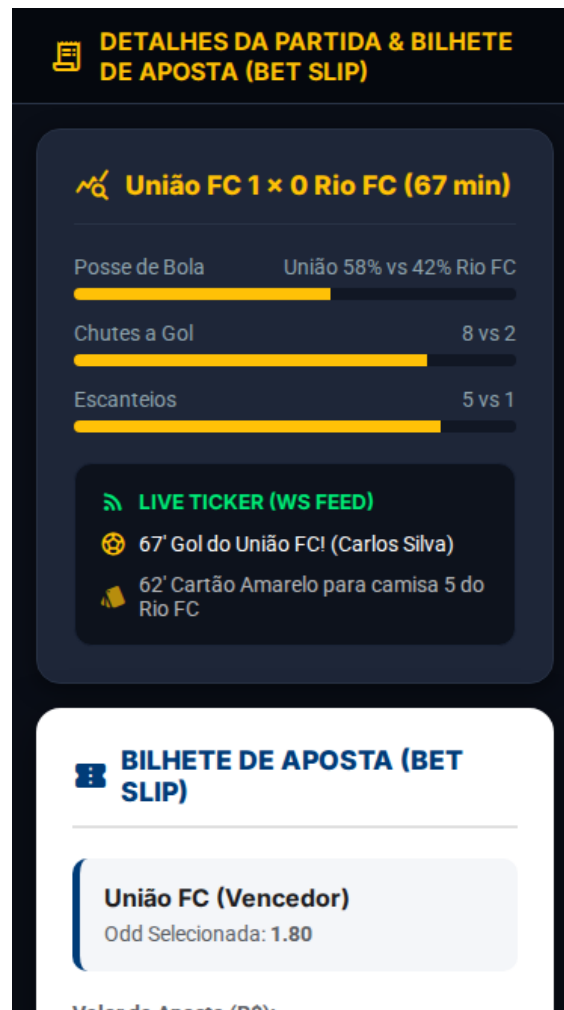


Figura 5: Protótipo de Bet Slip e Detalhes — Visão Mobile Responsiva.

6.2.1 História de Usuário (Scrum)

US-02: Configuração e Confirmação de Bilhete de Aposta

Como Apostador,

Eu quero inserir o valor da minha aposta (Stake) na caderneta lateral e ver o cálculo automático do retorno potencial em tempo real,

Para que eu possa revisar as condições do meu bilhete e registrá-lo instantaneamente via WebSocket.

6.2.2 Critérios de Aceite

- Calcular o retorno potencial ($R = \text{Valor} \times \text{Odd Total}$) instantaneamente ao digitar a quantidade.
- Transmitir a aposta via frame WebSocket PLACE_BET contendo o ID da partida e o palpite.

- Exibir modal de confirmação com código único do comprovante de aposta (*Ticket ID*).
- Deduzir o valor apostado do saldo da carteira do usuário imediatamente após o aceite do servidor.

6.2.3 Fluxo Feliz (Happy Path)

1. O apostador visualiza o palpite selecionado na caderneta de apostas (Bet Slip).
2. O apostador digita R\$ 50,00 no campo de entrada do valor.
3. O painel calcula e exibe em verde: **"Retorno Potencial: R\$ 97,50"**.
4. O apostador clica no botão principal **"Confirmar Aposta [WS]"**.
5. O navegador envia a mensagem WebSocket `{"type": "PLACE_BET", "matchId": 15, "odd": 1.95, "value": 50.00}`.
6. O servidor valida o saldo, registra o bilhete #1032 e devolve a confirmação com o carimbo de hora.

6.2.4 Fluxos Alternativos e Exceções

- **Fluxo Alternativo (Mudança de Odd no Aceite):** Se a odd mudar antes da confirmação final (ex: de 1.95 para 1.90), a tela exibe o alerta: *"[Odd alterada para 1.90. Aceitar nova cotação e confirmar?]"*.
- **Fluxo de Exceção (Saldo Insuficiente):** Se o valor digitado for maior que o saldo em conta, o botão de confirmação é desabilitado e a mensagem *"[Saldo insuficiente — Faça um depósito]"* é exibida.

6.2.5 Detalhamento de Botões e Componentes Interativos

Campo "Valor da Aposta (R\$)": Input numérico para digitação do montante a ser apostado.

Botão "Confirmar Aposta" (Verde Destaque): Envia o bilhete via WebSocket para validação.

Botão "Remover Palpite" (Ícone Lixeira): Cancela a seleção e limpa o Bet Slip.

6.2.6 Navegação entre Protótipos

A confirmação bem-sucedida do bilhete redireciona a aposta ativa para a **Tela 3 (Carteira e Gestão de Cash Out)** para acompanhamento e encerramento antecipado.

6.3 Tela 3: Carteira e Gestão de Cash Out (tela3-cashout.html)

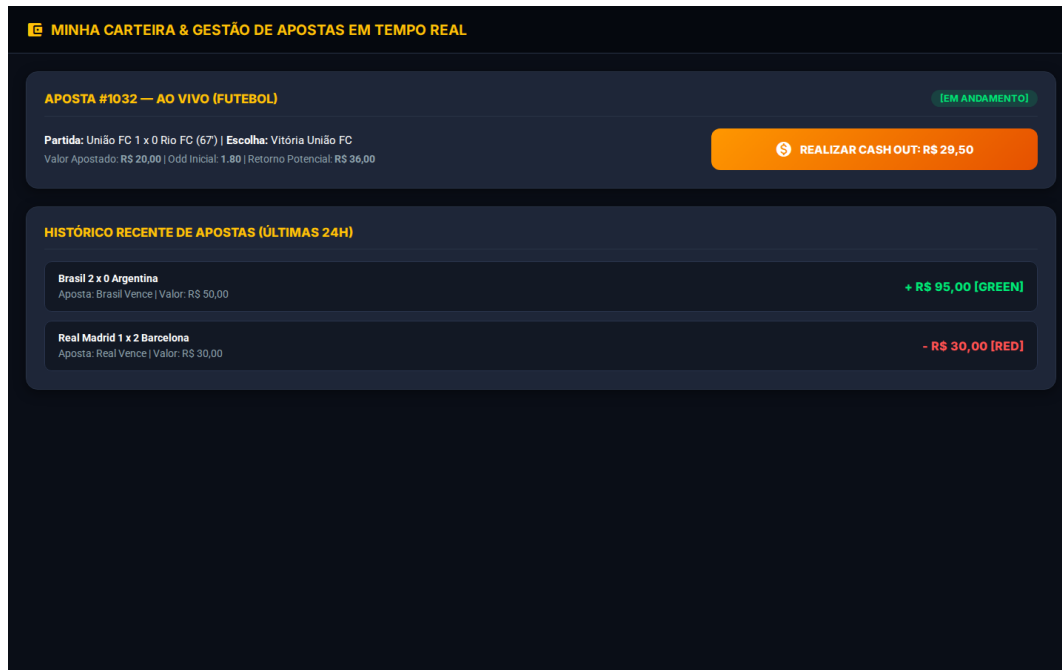


Figura 6: Protótipo de Carteira e Cash Out — Visão Desktop (Gestão Financeira).



Figura 7: Protótipo de Carteira e Cash Out — Visão Mobile Responsiva.

6.3.1 História de Usuário (Scrum)

US-03: Encerramento Antecipado de Apostas (Cash Out)

Como Apostador,

Eu quero visualizar minhas apostas ativas com a oferta de *Cash Out* recalculada dinamicamente via WebSocket segundo os acontecimentos do jogo,

Para que eu possa encerrar minha aposta antecipadamente, garantindo um lucro parcial ou reduzindo eventuais perdas.

6.3.2 Critérios de Aceite

- Exibir lista de bilhetes ativos com indicador visual de status em tempo real.
- Atualizar a proposta de valor do *Cash Out* instantaneamente via pacotes de atualização WebSocket.

- Permitir a execução do *Cash Out* com um clique no botão dedicado.
- Creditar o valor aceito no saldo da carteira imediatamente pós-confirmação do servidor.

6.3.3 Fluxo Feliz (Happy Path)

1. O apostador acessa a aba "Minhas Apostas" em `tela3-cashout.html`.
2. O apostador identifica o bilhete ativo #1032 da partida Flamengo x Palmeiras.
3. Como o Flamengo está vencendo o jogo por 1x0, a proposta de *Cash Out* sobe via WS para **R\$ 29,50**.
4. O apostador clica no botão **"Encerrar Aposta (R\$ 29,50)"**.
5. O navegador envia a mensagem WS `{"type": "CASH_OUT", "betId": 1032, "requestedValue": 29.50}`.
6. O servidor processa o encerramento, credita R\$ 29,50 na carteira e altera o bilhete para a aba de "Encerradas".

6.3.4 Fluxos Alternativos e Exceções

- **Fluxo Alternativo (Recusa de Cash Out por Mudança de Placar):** Se o time adversário empatar o jogo no momento do clique, o servidor recusa o Cash Out, exibe o aviso *"[Proposta expirada — Placar alterado]"* e recalcula o valor.
- **Fluxo de Exceção (Aposta Finalizada por Tempo):** Se a partida terminar normalmente sem Cash Out, a aposta é liquidada como Vencedora ou Perdida e atualizada no histórico.

6.3.5 Detalhamento de Botões e Componentes Interativos

Botão "Realizar Cash Out" (Verde/Laranja): Encerra antecipadamente o bilhete pelo valor cotado.

Filtros "Abertas / Encerradas": Alterna entre apostas ativas e histórico finalizado.

Display de Saldo da Carteira: Mostra o saldo atualizado disponível para novas apostas ou saque.

6.3.6 Navegação entre Protótipos

Após realizar o Cash Out e ver o saldo ser creditado, o apostador pode clicar no atalho superior para retornar ao **Lobby de Apostas (Tela 1)** para realizar novos palpites.

6.4 Mapeamento Consolidado do Fluxo de Navegação (Ws2)

O diagrama a seguir descreve a jornada do apostador e o fluxo de dados entre os protótipos da plataforma de apostas:

